

Final model · Stage 4: the scoring package

Raw application in → decision JSON out · Python 3.14 + scikit-learn 1.9.0 · validation data only · test still sealed

Result: one engine, **CreditRiskEngine**, takes a raw application and returns the decision JSON. **8 of 8 required checks pass**. The original raw CSV rows of 2,000 validation loans reproduce Stage 3's PD, band, decision and expected loss **exactly** (difference 0.0). Scoring one application takes about 0.4 s.

A. What is in the package

Everything the engine needs sits in one folder, *models/final/04_scoring_package/bundle/* (about 10 MB, now committed so the live demo can be deployed straight from GitHub). *manifest.json* stores each file's SHA-256 checksum and the library versions. The engine **refuses to start** if a file was changed or if scikit-learn is not 1.9.0, the version that fitted the preprocessor (the Stage 1–3 notebooks only read already-transformed data, so their scikit-learn version did not matter; the engine transforms raw input, so it does).

Step	File in bundle/	From	Size
1. Feature engineering, imputation, encoding	preprocessor_v2.joblib	Preprocessing V2 (fitted on train)	5.5 MB
2. Probability of default (146 features, 1,931 trees)	final_model.txt	Stage 2: M3-B-mono-nozip	4.8 MB
3. Calibration (Platt: 2 numbers)	calibrator.json	Stage 2	< 1 KB
4. Risk band, decision, expected loss	decision_policy.json	Stage 3	2 KB
5. Reason codes (SHAP → sentences)	code: reason_codes.py	Stage 2	–
Integrity	manifest.json	checksums, versions, features	–

B. What an application must contain

Only **loan amount** and **term** (36 or 60) are required. The model reads 65 raw fields; any not supplied get the training median or the "missing" encoding and are listed back in *missing_fields*, so the caller always knows what was assumed. 37 raw LendingClub columns are ignored: LendingClub's own grade, sub-grade, interest rate and instalment (version B), zip code (dropped in Stage 2), and joint-application fields. Impossible values (term 48, FICO 950, negative income, a malformed date) are rejected with a readable message.

Input group	Fields	Examples
loan	4	loan_amnt, term, purpose, title
applicant	7	annual_inc, dti, emp_title, emp_length, home_ownership, verification_status, addr_state
credit score / history	8	fico_range_low, earliest_cr_line, inq_last_6mths, mths_since_recent_inq, delinq_2yrs, pub_rec, pub_rec_bankruptcies, mths_since_last_delinq
other credit-bureau fields	46	mths_since_last_record, open_acc, revol_bal, revol_util, total_acc, collections_12_mths_ex_med, mths_since_last_major_derog, acc_now_delinq ...

C. Fresh-session test

Check (run as its own process: tests/engine_fresh_session_test.py)	Result	Detail
Loads in a fresh process; checksums match	PASS	4 files verified in 0.2 s; scikit-learn 1.9.0
A tampered bundle file is refused	PASS	calibrator edited → engine refuses to start
Raw CSV rows of 2,000 validation loans reproduce Stage 3	PASS	max PD difference 0.0; bands, decisions, EL identical
Single scoring = batch scoring (20 loans)	PASS	max difference 0.00005 (JSON rounds PD to 4 decimals)
Sparse hand-written application → complete JSON	PASS	PD 0.1008, APPROVE
Impossible inputs rejected with a message (6 cases)	PASS	e.g. term 48 → "term must be 36 or 60 (months)"
Same input → same output	PASS	
Higher FICO never raises PD; higher DTI never lowers it	PASS	FICO 640→850: PD 0.134→0.048; DTI 0→40: 0.076→0.191
one application scored in under 200 ms (median of 50)	WARNING	median 384 ms (warning only)

D. What the engine returns

A hand-written applicant (60 months, DTI 27.5%, FICO 665, revolving use 88%, 3 recent inquiries) who left 44 bureau fields blank:

```
{
  "probability_of_default": 0.6642,
  "risk_band": "R7", "risk_band_label": "severe risk",
  "decision": "DECLINE", "decision_rule": "PD >= 27.5%",
  "loan_amount": 15000, "term_months": 60,
  "expected_loss_usd": 6243.03, "expected_loss_rate": 0.4162,
  "reasons": [
    {"reason": "Loan term: 60 months", "impact": 0.6336},
    {"reason": "Job title (historical default rate of similar titles): 24.5%", "impact": 0.2277},
    {"reason": "Debt-to-income ratio: 27.5%", "impact": 0.2227},
    {"reason": "Credit score (FICO): 665", "impact": 0.2159}],
  "strengths": [...2 items...],
  "missing_fields": [...44 bureau fields...],
  "warnings": [...], "model": {"name": "M3-B-mono-nozip", "version": "1.0.0"}
}
```

Three real validation applicants, scored from their original raw CSV rows (▲ raises risk, ▼ lowers it)

APPROVE · PD 7.3% · R2 \$14,400, 36 months · EL \$536 <i>actually repaid</i> ▲ Age of oldest revolving account: 57 months ▲ Debt-to-income ratio: 23.2% ▲ Months since newest revolving account: 23 months ▼ Loan term: 36 months	REVIEW · PD 27.5% · R5 \$24,900, 60 months · EL \$4,284 <i>actually repaid</i> ▲ Loan term: 60 months ▲ Loan amount: \$24,900 ▲ Accounts opened in the last 24 months: 8 ▼ Annual income: \$130,000	DECLINE · PD 30.2% · R6 \$20,000, 60 months · EL \$3,782 <i>actually repaid</i> ▲ Loan term: 60 months ▲ Accounts opened in the last 24 months: 10 ▲ Loan amount: \$20,000 ▼ Total credit lines ever: 55
--	--	---

E. Speed (one application)

Step	ms
preprocessor: engineer	98
preprocessor: filter	3
preprocessor: encode	308
preprocessor: dedupe	0.5
LightGBM PD (1,931 trees)	1
SHAP values (pred_contrib)	11
calibration + policy + reasons	1
TOTAL engine.score()	380

Why 0.4 s?

scikit-learn's target encoder rebuilds its lookup table of 143,539 job titles and 31,812 loan titles on **every** call. That is fine for a web demo. If speed ever matters, the tables can be cached as a dictionary at start-up; we keep the exact scikit-learn path because it is proven to reproduce Stage 3 to the last digit. The model itself (1 ms) and its SHAP explanation (11 ms) are fast.

What we learned

- The engine is the same code path that produced every validation number in Stages 1–3: proven by re-scoring 2,000 loans from the untouched raw file with zero difference.
- Blank bureau fields are allowed but matter: the hand-written applicant's PD rests partly on assumed values. For the live demo (Stage 6), the form will start from complete real profiles, and the response will show *missing_fields*.
- Versions are pinned in *requirements-engine.txt*, so the Docker image for Stage 6 rebuilds exactly this environment.

Next: Stage 5, open the sealed 2015 H2 test set once and score it with this frozen bundle.